How The Apps Work Together:

A) Simplified to two main apps, logical grouping of related features
B) Users app handles authentication, profiles and user related functionalities
C) Projects app handles everything video-related

1."users" App (User Management)

```
users/
├── models.py        # User Profile model
├── views.py         # Login/Register logic
└── templates/       # User-related pages
```

Purpose-

1) Handles user accounts (signup/login)
2) Manages user profiles
3) Stores user information
4) Manages the 'user to user following' feature '
5) Handles authentication

Key Models- ***User , Profile , Follow***

2."projects" App (Core Features)

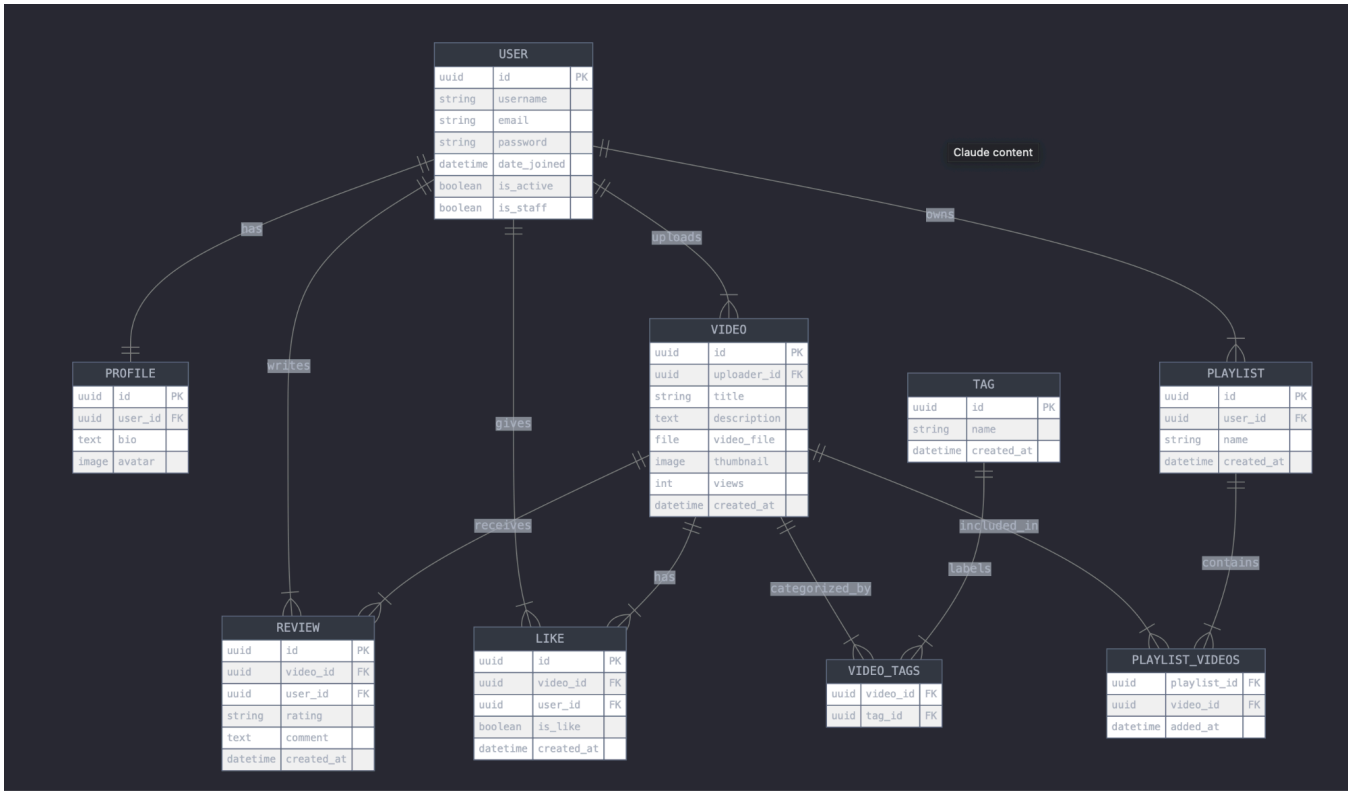
```
projects/
├── models.py        # Video storage/metadata
├── views.py         # Video upload/playback logic
└── templates/       # Video-related pages
```

Purpose-

1) Handles video uploads
2) Manages video playback
3) Stores video metadata
4) Manages video listings
5) Handles video reviews/ratings
6) Manages video tagging system
7) Thumbnail Processing

Key Models- ***Video, Tag, Like, Review, Playlist***

DB Schema



Key Relationships Between Tables:

1. User - Profile
- Practical: One user gets one profile, no more no less
- Technical: One-to-One relationship with Profile.user_id referencing User.id
- Example: Like your YouTube account and your YouTube channel info

2. User - Video
- Practical: One user can upload many videos, each video has one uploader
- Technical: One-to-Many relationship with Video.uploader_id referencing User.id
- Example: Like a YouTuber (user) uploading multiple videos to their channel

3. User - Review
- Practical: One user can write many reviews, each review comes from one user
- Technical: One-to-Many relationship with Review.user_id referencing User.id
- Example: Like when you comment on different YouTube videos

4. User - Like
- Practical: One user can like many videos, each like comes from one user

- Technical: One-to-Many relationship with Like.user_id referencing User.id
- Example: Like clicking the thumbs up button on multiple YouTube videos

5. User - Playlist
- Practical: One user can create many playlists, each playlist belongs to one user
- Technical: One-to-Many relationship with Playlist.user_id referencing User.id
- Example: Like creating multiple playlists of your favorite music videos

6. Video - Tags
- Practical: One video can have many tags, one tag can be on many videos
- Technical: Many-to-Many relationship using VIDEO_TAGS junction table
- Example: Like adding multiple tags (#funny, #music) to a video

7. Video - Playlist
- Practical: One video can be in many playlists, one playlist can have many videos
- Technical: Many-to-Many relationship using PLAYLIST_VIDEOS junction table
- Example: Like adding one music video to multiple different playlists

8. Video - Review
- Practical: One video can have many reviews, each review belongs to one video
- Technical: One-to-Many relationship with Review.video_id referencing Video.id
- Example: Like multiple users commenting on one YouTube video

9. Video - Like
- Practical: One video can have many likes, each like belongs to one video
- Technical: One-to-Many relationship with Like.video_id referencing Video.id
- Example: Like multiple users hitting like on the same video

GITHUB WORKFLOW

1.Initial Status and Feature Branch Work # Check current branches (*indicates current branch) > git branch # Stage all changes > git add . # Commit changes with message > git commit -m "your message" # Push feature branch to remote > git push origin feature/branch-name	2. Update Main Branch # Switch to main branch > git checkout main # Pull changes from develop to main > git pull origin develop # Push changes to main > git push origin main #If rejected: # sync with remote main > git pull origin main # try pushing again > git push origin main	3.Update Develop Branch # Switch to develop > git checkout develop # Get latest changes > git pull origin develop # Merge feature branch into develop > git merge feature/branch-name # Push updated develop > git push origin develop
4.Clean Up Feature Branch # Delete local feature branch > git branch -d feature/branch-name # Delete remote feature branch > git push origin --delete feature/branch-name	5.Start New Feature Branch # Create and switch to new feature branch from develop > git checkout develop > git checkout -b feature/new-branch-name # Push new branch to remote and set tracking > git push -u origin feature/new-branch-name	6.Final Sync # Get all remote updates > git fetch --all # Pull updates for all branches > git pull --all # Verify branch structure > git branch